

- has been written many years ago. Given the advent of non-volatile byte addressable memory, you have been given the task of evaluating which software applications need a rewrite to take advantage of it. How would you evaluate whether an application needs a rewrite to take advantage of non-volatile byte addressable memory?
12. You are given a computer system whose architectural parameters you are not aware of. You are not told what kind of CPU it is, what its frequency is, or the sizes and access times of the different cache hierarchies it has. You can write software programs and find out this information. Explain your approach to identifying the cache line sizes, cache sizes and latencies through experimentation.
 13. What are context-free grammars? Would you be able to use a context-free grammar to parse 'C' programming language? Would you be able to build a context-free grammar to parse the English language?
 14. You have been told that an application potentially contains a memory corruption bug. You are tasked with identifying it. A colleague says that he has given the application 10,000 runs and none of the runs hit the memory corruption issue. Is that enough evidence to conclude that there is no memory corruption issue? What would you do to conclusively establish the presence or absence of the memory corruption bug? (*Hint: Are you familiar with the 'black swan' phenomenon? If I tell you that "All swans are white," then if you observe a million white swans, does that establish the fact that all swans are white? On the other hand, what would happen if you see even one black swan?*)
 15. Your company has a legacy software application that was written and deployed on 32-bit operating systems. Now you need to migrate it to an environment where it needs to be deployed in a 64-bit operating system. How would you assess the code changes needed to migrate it to the 64-bit platform?
 16. You are coding a concurrent application in 'C' on Linux. You need to enforce mutual exclusion in accessing a data structure. You have, at your disposal, a number of synchronisation mechanisms like Spinlock, reader-writer lock or read-copy-update mechanisms. How would you decide what synchronisation mechanism to use?
 17. You are given a single-threaded application written in 'C'. Given that it needs to be employed on multi-core chips, you are asked to investigate whether it is possible to parallelise it into a concurrent application. How would you determine whether it would be profitable to parallelise it without actually creating the parallelised version and running it? *Hint: Are you familiar with Amdahl's law?*
 18. What are the techniques for verifying the integrity of

data storage systems? Are you familiar with checksumming techniques? What kind of architectural support is available to enable faster computation of checksums?

19. What are key-value stores? Where do you see them being used widely?
20. Are you familiar with the CAP theorem? Is it possible for a distributed system to satisfy all the requirements of consistency, availability and partition tolerance? What is eventual consistency? Can you give an example of a large scale distributed system which is eventually consistent?

My 'must-read book' for this month

This month's suggestion comes from S Nagarajan and he recommends the book 'Java Coding Guidelines: 75 Recommendations for Reliable and Secure Programs' by Fred Long, Dhruv Mohindra, Robert C Seacord, et al. For each coding guideline for secure and reliable programming, they provide non-compliant code examples and then show how to change the code to conform to the guideline. Thank you SN, for your book recommendation. For 'C' programmers, a related book of interest would be 'Secure Coding in C and C++' by Robert C Seacord. This book describes the commonly exploited software vulnerabilities and how to write code to avoid them.

I also wanted to tell our readers of the new course on 'Automata' currently being offered on Coursera (www.coursera.org), by Jeff Ullman. There is a free online textbook: 'Foundations of Computer Science' written by Al Aho and Prof Ullman, available at <http://i.stanford.edu/~ullman/focs.html>. This is definitely a course worth taking.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in this column. It would help many readers who want to improve their software skills.

If you have any favourite programming questions or software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you a great and happy 2014! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multicore and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=HYPERLINK&http://www.linkedin.com/groups?home=&gid=2339182&HYPERLINK&http://www.linkedin.com/groups?home=&gid=2339182>